Pyrite: Byte-Neutral Group-Size Trading for 2-Bit Quantization of 27B–33B Language Models on 16 GB Apple Silicon

Anonymous Authors
Paper under double-blind review

Abstract

Running 27B–33B parameter language models on a 16 GB unified-memory laptop requires roughly 2-bit weights, at which point quality degrades sharply. We describe *Pyrite*, a simple, kernel-free recipe for standard affine group quantization (as implemented in Apple MLX) that redistributes bits *without changing the total tensor size by a single byte*. The recipe widens the quantization group of the SwiGLU `gate_proj`/`up_proj` matrices from 64 to 128 and reinvests exactly the freed bytes to upgrade a sensitivity-ranked subset of projections from 2-bit to 3-bit. Because the cost of each move is an integer multiple of a common unit, any allocation satisfying $4n_{\text{down}} + 5n_{\text{gate/up}} = 2L$ (for L layers) yields a byte-identical model. On a sparse Mixture-of-Experts model (llm-jp-4-32b-a3b-base) the best configuration reduces held-out Japanese KL divergence to the BF16 teacher by 11.0% (English: 12.9%), and on a dense hybrid model (Qwen3.8-27B) by 22.3% (English) and 22.4% (Japanese), relative to uniform 2-bit quantization of identical size. In both architectures, allocations that mix `down_proj` upgrades with a small number of late-layer `gate`/`up` upgrades outperform allocations that upgrade `down_proj` only. All profiling and evaluation, including BF16 teacher references for 52–60 GB checkpoints, run on a single 16 GB M2 Pro by streaming one layer at a time. The resulting models load with unmodified `mlx-lm`.

1 Introduction

Consumer laptops with 16 GB of unified memory are a common deployment target for local language models. A 32B-parameter model needs about 60 GB in BF16 and about 16 GB even at 4 bits, so fitting it with headroom for the OS and KV cache requires an average of roughly 2.5 bits per weight. At this rate uniform group quantization loses substantial fidelity.

Mixed-precision quantization is the standard remedy, but most practical schemes either change the total size of the model (making comparisons against a uniform baseline unfair) or require custom kernels and file formats. We ask a narrower question: *holding the tensor byte count exactly fixed and using only the stock affine quantization kernels of MLX, how much fidelity can be recovered by re-allocating bits across the MLP projections?*

Our contributions are:

1. A byte-neutral reallocation rule for affine group quantization: widening the `gate`/`up` groups from 64 to 128 frees precisely enough bytes to upgrade half of the layers' `down_proj` (or a mix of `down` and `gate`/`up`) from 2 to 3 bits, with zero byte difference from the uniform baseline (§3).

2. An empirical comparison, at identical byte budgets, on a sparse MoE model and a dense hybrid model, in English and Japanese, showing consistent gains and a consistent preference for mixed allocations (§5).

3. A single-pass, layer-streaming profiler and teacher-reference pipeline that runs end-to-end on a 16 GB machine for checkpoints several times larger than memory (§3.3).

We release the quantized models and code. We emphasise that the individual ingredients—sensitivity-aware mixed precision, the outlier-prone nature of `down_proj`, and the group-size/accuracy trade-off—are known; our contribution is their combination into an exactly byte-neutral recipe and its cross-architecture evaluation under a strict equal-size protocol.

2 Related Work

Post-training quantization of LLMs. GPTQ [Frantar et al., 2023] and AWQ [Lin et al., 2024] established 3–4 bit weight-only quantization with group-wise scales; both report the accuracy/size trade-off of group size. Extreme 2-bit methods such as QuIP# [Tseng et al., 2024] and AQLM [Egiazarian et al., 2024] use vector/lattice codebooks and dedicated kernels. Our work stays within plain affine group quantization so that the output runs on unmodified MLX kernels.

Mixed precision and sensitivity. Hessian-based bit allocation [Dong et al., 2019] and sensitivity-weighted non-uniform schemes such as SqueezeLLM [Kim et al., 2024] and SpQR [Dettmers et al., 2024] allocate precision by importance. Practical toolchains such as llama.cpp's k-quants assign higher precision to selected tensors (often including `down_proj`) heuristically, typically at a net size increase.

Outliers. Activation outliers in transformer FFNs [Dettmers et al., 2022] and very large individual activations/weights [Sun et al., 2024, Yu et al., 2024] explain why the projection consuming the SwiGLU product [Shazeer, 2020], `down_proj`, is particularly sensitive.

3 Method

3.1 Byte accounting for affine group quantization

MLX affine quantization with b bits and group size g stores b bits per weight plus one BF16 scale and one BF16 bias per group, i.e. $b + 32/g$ bits per weight. Hence 2-bit/g64 costs 2.50, 2-bit/g128 costs 2.25, and 3-bit/g64 costs 3.50 bits per weight.

In a SwiGLU MLP, `gate_proj`, `up_proj` and `down_proj` each contain the same number n of weights ($d \times d_{\text{ff}}$; for MoE, $E \times d \times d_{\text{ff}}$ per layer). Let $u = n/4$ bits ($n/32$ bytes) be one *unit*. Then

- widening one `gate` or `up` matrix from g64 to g128 frees $0.25n = 1$ unit;
- upgrading one `down_proj` from 2-bit/g64 to 3-bit/g64 costs $1.00n = 4$ units;
- upgrading one widened `gate`/`up` matrix from 2-bit/g128 to 3-bit/g64 costs $1.25n = 5$ units.

Widening every `gate`/`up` matrix in L layers frees $2L$ units, so any allocation with

$$4\, n_{\text{down}} + 5\, n_{\text{gate/up}} = 2L \tag{1}$$

produces a model whose tensor byte count equals the uniform 2-bit/g64 baseline exactly (provided d and d_{ff} are divisible by 128). With $n_{\text{gate/up}} = 0$ this upgrades exactly $L/2$ `down_proj` matrices. For llm-jp-4 ($L = 32$) the allocations used are $16{\cdot}4$ and $11{\cdot}4 + 4{\cdot}5$; for Qwen3.8-27B ($L = 64$), $32{\cdot}4$ and $22{\cdot}4 + 8{\cdot}5$. All other tensors (embeddings, attention, LM head, router) are quantized identically in the baseline and in every candidate, and we verify byte equality programmatically for every model we evaluate.

Table 1: Models and exact tensor byte budgets (identical for baseline and all candidates of a model).

Model	Type	Layers	Params	Budget (bytes)	GiB
llm-jp-4-32b-a3b-base	MoE (128 experts, top-8)	32	32.1B	10,533,950,464	9.81
Qwen3.8-27B	Dense (hybrid DeltaNet)	64	27B	10,854,616,576	10.11
DeepSeek-Coder-V2-Lite	MoE (64 routed, MLA)	27	15.7B (2.36B act.)	5,241,265,152	4.88

3.2 Sensitivity scoring

Candidates are ranked by the reduction in error obtained by the upgrade. For llm-jp-4 we used a composite per-expert score combining routing frequency, an estimated output norm and the reduction in weight-space Frobenius error, summed over experts (calibration: Japanese text). For the dense models we used the activation-weighted output MSE $\mathbb{E}\|XW^\top - X\hat{W}^\top\|^2$ on 2,048 calibration tokens (1,024 English, 1,024 Japanese), multiplied by a mild depth prior $1 + 0.5\,\ell/(L-1)$. For each model we then evaluate a small number of allocations satisfying Eq. (1) and select the winner on a calibration-validation split, never on held-out data.

3.3 Single-pass layer streaming on 16 GB

The BF16 checkpoints (about 52 GB for Qwen3.8-27B and 60 GB for llm-jp-4) do not fit in memory. We download shards on demand, load one decoder layer at a time, and in a single pass (i) propagate BF16 hidden states for all calibration and evaluation sequences, (ii) measure the projection-level errors, (iii) store every candidate quantization of the layer's MLP (2b/g64, 2b/g128, 3b/g64) to a cache, and (iv) delete shards that are no longer needed. After the last layer we compute the BF16 teacher's logits, storing the top-128 log-probabilities plus the residual tail mass per position. Candidate models are then assembled from the cache in seconds without touching the BF16 weights again.

4 Experimental Setup

Models. Table 1. Embeddings, attention projections and LM head are 4-bit/g64 in all configurations.

Metrics. Our primary metric is the KL divergence from the BF16 teacher to the quantized student, computed exactly over the teacher's top-128 tokens plus a tail bucket (`kl_top128_tail`); we also report perplexity and top-1 agreement with the teacher. Held-out English and Japanese Wikipedia passages are used for evaluation and are never used for allocation or model selection.

Protocol differences. The llm-jp-4 experiments (performed first) evaluate the first 256 tokens of each passage (32 JA / 16 EN passages). The Qwen3.8 experiments concatenate passages and split them into 256-token windows (16 EN / 16 JA windows; validation: 8+8 windows). Numbers are therefore comparable within a model but not across models.

5 Results

MoE (Table 2). Upgrading 16 `down_proj` layers reduces Japanese held-out KL by 7.4%. The mixed allocation (11 `down` + 4 `gate`/`up`, all in layers 21–31) reduces it by 11.0% and English KL

Table 2: llm-jp-4-32b-a3b-base (MoE), 10,533,950,464 bytes for every row. Percentages are relative to the uniform baseline.

	JA held-out			EN held-out		
Configuration	KL↓	PPL↓	Top-1↑	KL↓	PPL↓	Top-1↑
BF16 teacher	–	7.45	–	–	3.40	–
Uniform 2-bit/g64 (baseline)	0.358	10.84	72.2	0.581	5.83	71.9
Pyrite down-only (16 down)	0.332 (-7.4%)	10.46	72.9	0.516 (-11.2%)	5.45	72.8
Pyrite mixed (11 down + 4 gate/up)	0.319 (-11.0%)	10.35	73.2	0.506 (-12.9%)	5.42	74.2

Table 3: Qwen3.8-27B (dense), 10,854,616,576 bytes for every row. Val = calibration-validation split used for selection.

	EN held-out			JA held-out			Val
Configuration	KL↓	PPL↓	Top-1↑	KL↓	PPL↓	Top-1↑	KL↓
BF16 teacher	–	6.15	–	–	10.15	–	–
Uniform 2-bit/g64 (baseline)	0.623	10.92	67.2	0.586	18.35	66.0	0.611
Pyrite down-only (32 down)	0.516 (-17.2%)	9.89	70.1	0.513 (-12.5%)	16.57	67.9	0.498
Pyrite mixed (22 down + 8 gate/up)	0.484 (-22.3%)	9.60	70.8	0.454 (-22.4%)	15.55	68.7	0.445

by 12.9%, at an identical 9.8105 GiB.

Coding MoE (Table 4). On DeepSeek-Coder-V2-Lite (MLA attention, 64 routed experts + 2 shared experts), widening all 52 MoE `gate`/`up` projections from $g64 \to g128$ yields 52 budget units. Reinvesting this into upgrading 13 `down_proj` layers (layers 14–26) from 2-bit to 3-bit/g64 strictly preserves the 4.8813 GiB budget ($\Delta = 0$ B) while reducing multi-domain validation KL from 0.658 to 0.641 and raising top-1 token agreement to 70.0%. At 4.88 GiB, this leaves > 10 GiB of unified memory free on a 16 GB Mac for long-context code generation.

Dense (Table 3). Upgrading 32 `down_proj` layers reduces English/Japanese held-out KL by 17.2%/12.5%. The mixed allocation (22 `down` + 8 `gate`/`up` in layers 58–63) reduces them by 22.3%/22.4%. Upgraded slots: L35.down, L36.down, L44.down, L45.down, L46.down, L47.down, L48.down, L49.down, L50.down, L51.down, L52.down, L53.down, L54.down, L55.down, L56.down, L57.down, L58.down, L58.gate, L59.down, L59.gate, L60.down, L60.gate, L61.down, L61.gate, L61.up, L62.down, L62.gate, L62.up, L63.down, L63.gate.

Where the error is. Table 5 shows per-layer output MSE for Qwen3.8-27B. Error grows with depth for all projections, but much faster for `down_proj` (195× from layer 32 to 63) than for `up`/`gate` (5.2×/4.9×). Note that absolute MSE values are not directly comparable across projections, which have different output dimensionality and scale; the ranking in Eq. (1) therefore uses error *reduction* per projection type. We hypothesise that the mixed allocation wins because the marginal benefit of the last few `down_proj` upgrades (mid-depth layers with small error) is lower than that of upgrading late `gate`/`up` matrices, but we have not isolated this effect.

Runtime. All models load with stock `mlx-lm`. Qwen3.8-27B-Pyrite decodes at 12.16 tok/s on an M2 Pro (16 GB) in our measurement script, with a measured peak of 15.29 GiB including cache, i.e. very close to the device limit.

Table 4: DeepSeek-Coder-V2-Lite-Instruct (MoE, 15.7B total, 2.36B active), 5,241,265,152 bytes for every row (4.8813 GiB). Val = calibration validation split.

	Code held-out		EN held-out		JA held-out		Val
Configuration	PPL↓	Top-1↑	KL↓	Top-1↑	KL↓	Top-1↑	KL↓
BF16 teacher	1.59	–	–	–	–	–	–
Uniform 2-bit/g64 (baseline)	1.83	92.7	0.679	67.1	0.667	70.6	0.658
Pyrite down-only (13 down)	1.90	92.4	0.684 (+0.7%)	67.3	0.675 (+1.1%)	70.3	0.641 (-2.6%)
Pyrite mixed (8 down + 4 gate/up)	1.87	92.7	0.705 (+3.9%)	66.5	0.657 (-1.5%)	71.0	0.643 (-2.3%)

Table 5: Qwen3.8-27B activation-weighted output MSE at the baseline precision of each projection (selected layers).

Layer	down (2b/g64)	up (2b/g128)	gate (2b/g128)
L0	0.0001	0.0012	0.0014
L16	0.0012	0.0148	0.0162
L32	0.0061	0.0434	0.0544
L48	0.0164	0.0698	0.0781
L58	0.1109	0.2071	0.2677
L60	0.1637	0.2255	0.2787
L62	0.3231	0.2447	0.2879
L63	1.1832	0.2259	0.2668

6 Limitations

- **Small evaluation sets.** Each split has 4k–8k predicted tokens and we report single runs; we have not estimated variance.
- **Proxy metrics only.** We report KL, perplexity and top-1 agreement against the teacher, not downstream task accuracy (e.g. JGLUE, MMLU).
- **No external baselines.** We compare only against uniform 2-bit/g64 MLX quantization of equal size, not against GPTQ, AWQ, QuIP#, AQLM or llama.cpp k-quants.
- **Different scoring and protocols across models,** as described above; cross-model comparisons of magnitude are not meaningful.
- **Few candidate allocations** were evaluated per model; the reported winners are not optimal solutions of Eq. (1).

7 Conclusion

Widening `gate/up` quantization groups and reinvesting the exact freed bytes into 3-bit projections is a simple, byte-neutral and kernel-free way to improve 2-bit MLX models. On one MoE and one dense model it reduced teacher KL by 11–22% at identical size, with mixed `down+gate/up` allocations consistently preferred over `down`-only.

Availability

The quantized models are publicly available on Hugging Face:

- `https://huggingface.co/masahiroid/llm-jp-4-32b-a3b-base-Pyrite-2bit-MLX`
- `https://huggingface.co/masahiroid/Qwen3.8-27B-Pyrite-2bit-MLX`

All evaluation numbers in this paper are generated directly from the experimental result files by `paper/build_numbers.py`.

References

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefler, and Dan Alistarh. SpQR: A sparse-quantized representation for near-lossless LLM weight compression. In *International Conference on Learning Representations (ICLR)*, 2024.

Zhen Dong, Zhewei Yao, Amir Gholami, Michael W. Mahoney, and Kurt Keutzer. HAWQ: Hessian AWare quantization of neural networks with mixed-precision. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. Extreme compression of large language models via additive quantization. In *International Conference on Machine Learning (ICML)*, 2024.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023.

Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W. Mahoney, and Kurt Keutzer. SqueezeLLM: Dense-and-sparse quantization. In *International Conference on Machine Learning (ICML)*, 2024.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Mingjie Sun, Xinlei Chen, J. Zico Kolter, and Zhuang Liu. Massive activations in large language models. In *Conference on Language Modeling (COLM)*, 2024.

Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. QuIP#: Even better LLM quantization with hadamard incoherence and lattice codebooks. In *International Conference on Machine Learning (ICML)*, 2024.

Mengxia Yu, De Wang, Qi Shan, Colorado Reed, and Alvin Wan. The super weight in large language models. *arXiv preprint arXiv:2411.07191*, 2024.